```
    </div>
  )
}

export default SearchForm
```

It pulls the value of the form from the global state via the useAppSelector call that pulls the textFilter value, then calls a Redux thunk called search when the value changes:

appendix_react/01/app/javascript/contexts/schedule_context.ts

```
export const search = (searchTerm: string): ScheduleThunk => {
  return async (dispatch) => {
    let concerts: Concert[] = []
    if (searchTerm !== "") {
      const response = await fetch(`/concerts.json?query=${searchTerm}`)
      const data = await response.json()
      concerts = data.concerts
    }
    dispatch({
      type: "updateTextFilter",
      results: concerts as Concert[],
      text: searchTerm,
    })
  }
}
```

The pattern here is similar to the thunk that fetches the initial data. It makes a fetch call to the same React API (again, we've added a json path that returns the set of found concerts as JSON data, then passes that data along to the reducer to update the state):

appendix_react/01/app/javascript/contexts/schedule_context.ts

```
case "updateTextFilter": {
  return {
    ...state,
    textFilter: action.text,
    searchResults: action.results,
  }
}
```